

RESEARCH ARTICLE

What Gets Measured Gets Improved: Monitoring Machine Learning Applications in Their Production Environments

DOMINIK PROTSCHKY^{1,2,3}, LUIS LÄMMERMANN^{2,4},
PETER HOFMANN^{1,4}, AND NILS URBACH^{1,2,3}

¹Branch Business and Information Systems Engineering, Fraunhofer Institute for Applied Information Technology (FIT), 95444 Bayreuth, Germany

²University of Bayreuth, 95447 Bayreuth, Germany

³FIM Research Center for Information Management, 95444 Bayreuth, Germany

⁴appliedAI, 81671 Munich, Germany

Corresponding author: Dominik Protschky (dominik.protschky@fit.fraunhofer.de)

ABSTRACT Machine learning (ML) applications face many new, hardly predictable aspects in their production environments. Detecting new aspects in an ML production environment and understanding their impacts on the ML application is crucial if organizations are to ensure ML applications functionality. A monitoring entity is essential if one is to monitor ML applications in their production environments, to both continually minimize risks and improve ML application's performance. But existing monitoring approaches are struggling to deal with specifics that arise from ML applications. We aim at deriving monitoring practices and providing a holistic view over required steps in successful ML applications monitoring. Since there has been little research on this topic, we followed a qualitative research approach, i.e., we conducted an interview study combined with a multivocal literature review. Thus, we provide a theoretical framework of an ML-enabled agent in its production environment, five characteristics of ML applications' production environments and 17 monitoring practices – 14 practices arranged sequentially on a typical quality management cycle and three cross-sectional practices. To outline the ML specifics that arise in monitoring ML applications, we investigate the five ML production environment characteristics' influences on the ML monitoring practices.

INDEX TERMS Machine learning, MLOps, monitoring, software engineering.

I. INTRODUCTION

Machine learning (ML), as the primary technology behind artificial intelligence (AI), is seeing increasing adoption and commercialization [1], [2], [3]. Organizations are increasingly transitioning ML's promised potential from proofs-of-concept to market-ready ML products, i.e., into production environment. However, putting ML applications into production also introduces complex challenges for organizations [4], [5], [6], [7], [8]. For instance, many organizations using ML applications report issues like inaccuracy, hallucinations, and biases in their models, particularly in customer-facing applications [9], [10]. Recent surveys

indicate that about 23% of companies operating generative ML applications have experienced negative consequences due to inaccuracies [10]. As these issues are often unforeseeable prior to deployment, an organization's ML operations should be adaptable to changes through the environment [11]. Therefore, comprehensive monitoring of ML applications plays a key role in discovering and mitigating issues around their operation [4], [8], [12], [13], [14]. From an organizational perspective, robust monitoring practices enable organizations to promptly identify and address issues such as model drift, bias, performance degradation, and compliance risks, which can have significant organizational consequences [15], [16], [17], [18]. Without adequate monitoring, organizations risk deploying models that may produce inaccurate predictions or fail to meet regulatory requirements, leading

The associate editor coordinating the review of this manuscript and approving it for publication was Wai-Keung Fung¹.

to customer dissatisfaction, legal penalties, and reputational damage [19]. Therefore, implementing comprehensive ML monitoring practices is essential for organizations to ensure the reliability, accountability, and continued value generation of their ML applications in production environments [4], [13], [20], [21], [22].

However, performing comprehensive ML monitoring within organizations is a complex endeavor, as ML applications are constantly evolving in terms of their underlying technology stack and capabilities, while continuously being exposed to new environmental circumstances [5], [6], [23]. Comprehensive approaches for ML monitoring are scarce. One established approach that explicitly considers ML monitoring is MLOps [19], [24]. MLOps builds on the principles of DevOps but incorporates specific practices for managing the entire ML lifecycle, including data processing, model training, deployment, and continuous monitoring [19], [24]. While literature on MLOps offers a structured approach to handle ML workflows, it focuses primarily on the overarching automation and management of ML workflows, lacking in-depth guidance for monitoring issues specific to ML models. Accordingly, literature about MLOps serves as a useful starting point, but falls short in providing organizations with actionable insights for complex, context-specific ML problems that arise in their production environment.

For organizations, it remains an open question which practices are required when monitoring ML applications in their production environments. Despite the growing recognition of the importance of ML monitoring, there is limited research that systematically identifies and categorizes effective monitoring practices. Owing to the insufficient conceptualization of ML monitoring practices and the wide range of ML challenges in production, we ask:

What are relevant practices for monitoring ML applications in their production environments?

To answer this research question, we followed a qualitative research approach [25], relying on intelligent agent theory [26], [27], [28]. First, we conducted a multi-vocal literature review to gain initial understanding of the current state of research on ML monitoring following Ogawa and Malen [29] as well as Garousi et al. [30]. Second, we conducted an interview study with practitioners to gain further insights into monitoring practices for ML applications. Third, we reviewed relevant ML monitoring tools and products to expand our insights into real-world monitoring practices.

Our results make both theoretical and practical contributions: First, we contribute to theory by proposing five environmental characteristics that affect the monitoring of ML applications, as well as 17 practices for effective ML monitoring. We further contribute to practice by providing a monitoring framework that will guide ML engineers in comprehensively monitoring ML applications. Overall, this paper contributes to the ongoing discourse on successfully operating and managing AI applications in their production environment [23], [31], [32].

II. THEORETICAL FOUNDATIONS

A. INTELLIGENT AGENTS IN THE MACHINE LEARNING CONTEXT

To conceptualize the specifics of ML monitoring, we refer to intelligent agent theory as our theoretical lens, capturing the ML-based agent's inherent characteristics and behavior [26], [27], [28]. The theoretical foundations of intelligent agents and ML build a solid basis for understanding ML applications in their production environments for two reasons: First, the strong focus on the interactions between an agent and its environment helps one to consider agents' ability to actively participate in their production environments [1]. Second, both the agent concept and ML refer to performance measurement as a key aspect [26], [33].

An intelligent agent is situated in an environment where it can perceive aspects of the environment through sensors and respond to stimuli through actuators [26], [28], [34]. To be considered as an intelligent agent assumes the following characteristics: An agent's actions should be reactive (able to react to changes in the environment), proactive (able to continually seek to achieve its goals), autonomous (able to learn as much as possible by itself to make its own decisions), capable of learning (able to learn from the perceived input and to improve over time), connected to its environment (able to constantly interact with the environment), and rational (able to select an action that optimizes the performance measure in question) [26], [27], [28], [34], [35], [36]. For the intelligent agent's function [26], we refer to ML as a set of concepts that enables agents to learn from experience (i.e., data) to solve a specific task rather than being explicitly programmed [2], [33], [37]. Contemporarily, ML is the prevailing AI technology [2], [38] that can enable agents to act intelligently [26].

Deployed ML-enabled agents are typically embedded in a production environment (e.g., business process) in order to contribute value with their agentic capabilities, for instance, by analyzing data, automating routine tasks, or providing decision support. In doing so, ML-enabled agents are constantly retrieving data from their production environment, processing available information and respond to their environment. Accordingly, the production environment of ML-enabled agents significantly affects the agents' behavior requiring close consideration during monitoring. Drawing on existing research, we recognize five prevailing characteristics conceptualizing the ML-enabled agent's production environment (Table 1).

Building on our understanding of an ML-enabled agent and their production environments, we derive the following framework of an ML application in a production environment (Figure 1) with two layers: real-world layer and representation layer.

We define the representation layer as a conceptual layer that models the real-world layer and hence describes how those real-world phenomena are represented in a more subjective, symbolic, or formal way. The representation layer could refer to the way real-world data is encoded into algorithms, databases, or models. In the representation layer, the

TABLE 1. Overview over ML applications' production environment characteristics.

Characteristics	References
C1 - Data representation: An ML application's (training) data is just a representation of the real world and does not contain every piece of information from it.	4, 12, 18, 40–46
C2 - Metrics proxy: An ML application's metrics are incomplete and are only a proxy for aspects in the real world.	4, 43, 44, 47–54
C3 – Assumptions: An ML application contains several assumptions that may not be appropriate for the real world.	18, 42, 43, 48, 50, 55–58
C4 – Changes: An ML application's production environment changes over time owing to changes in the real world.	4, 12, 17, 40–43, 48, 50, 59–62
C5 – Entanglement: An ML application's high entanglement with itself and its environment leads to many dependency types.	12, 22, 40, 49, 55, 59, 60, 63–65

numbering (1 to 8) stands for core measurement areas in the ML application's main parts: The core ML application includes data, data preprocessing, ML algorithm(s), and the output (2, 3, 5, 6) [2], [65], and it runs on appropriate technical infrastructure (4). An ML application should perform a specific task that determines the relevant factors that may affect the application (7). How well (depending on an organization's business metrics) an ML application performs its tasks, is reflected in the ML application's business value for the organization (8). ML applications may also interact with other agents (e.g., humans or other ML-enabled agents) to solve their tasks (1). The framework helps one to understand an ML application's representation (the representation layer) and its connection to the real world (the real-world layer).

We define the real-world layer as an accumulation of observable and objective things [66], [67]. Such things can relate to one another and be part of a system that is defined by different values and goals [68]. According to our definition, an ML application can perceive these things and can adapt to them over time when they differ from the ML application's knowledge [26]. However, the real world is highly complex and hardly entirely describable. Russell and Norvig [26] provided a few dimensions (e.g., discrete vs. continual and deterministic vs. stochastic) that can help to better determine an environment's properties and incorporate them in an ML application's design. In an ML context, the real-world layer contains an ML application's production environment and determines the so-called ground truth [58]. However information from the real-world layer – such as the ground truth label – is not always available in the production environment. For instance, the ground truth label for predicting a credit

misuse may not result until months later, by which time the credit misuse has already occurred.

The representation layer implies that an ML application will always be a representation of the real world. However, this representation is constrained because not all information about the real world is available or can be depicted. Thus, not only known aspects (e.g., data or features) from the real world but also assumptions about the real world describe an ML application's design. We note that there are several relationships between representation and the real-world layer, for instance, data, which comes from the real world and goes into the ML application. A further connection between the real-world layer and the representation layer is an ML application's interaction with a further ML application, a rules-based computer program, or a human. Since ML applications do not only represent a part of the real world but actively participate in the real world, the ML application also affects the real world [1].

This is where monitoring comes in as entity that is able to observe the real-world layer through the representation layer.

This means that information from the real-world layer goes into the observer layer via the representation layer, but only under the restriction of the ML applications' production environment characteristics. One can observe an ML application in the representation layer via different metrics, but these metrics are only a proxy (C2) for different aspects of interest (e.g., accuracy for performance). As described above the characteristics data representation (C1), assumptions (C3), and entanglement (C5) lead to a certain disconnection between the representation and the real-world layer. In addition to C1-C3, the characteristic changes (C4) leads to a constantly changing status of C1-C3. Hence, the characteristics C1-C4 also complicate observing the representation layer and its real-world layer.

B. MACHINE LEARNING MONITORING

In general, governing information systems and software applications is a well-known and investigated terrain in IS research and practice. In the context of ML governance, organizations need to establish practices that ensure that their ML applications sustain and extend the organization's strategies and objectives [69]. Among these practices, monitoring combines the continual performance measurement of several properties with the analysis of the measurements to gain useful insights for tackling different issues, minimizing risks, and maximizing the software applications' added value [70], [71], [72], [73]. However, practitioners have recognized that monitoring practices from software monitoring or DevOps are insufficient for ML monitoring owing to new challenges that arise mainly from data and new algorithms in ML applications [12], [48], [58], [59]. First, mismatches between training and production data lead to a situation where not all information is captured in the training phase (see also C1 and C3). Second, ML mainly exhibits a nonmonotonic error propagation, which makes selecting the appropriate metrics

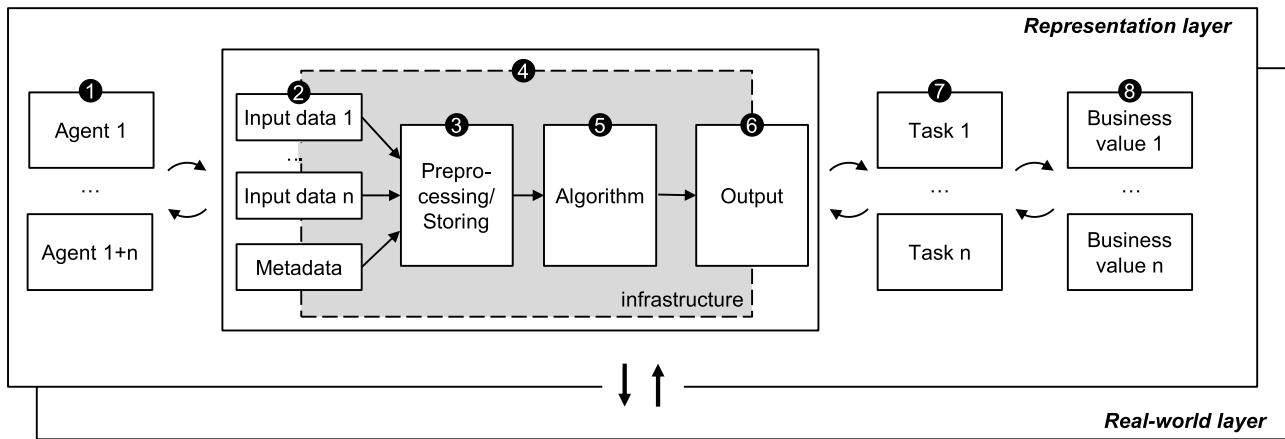


FIGURE 1. A machine learning application in its production environment.

a nontrivial task (see also C2). Third, hidden feedback loops in ML applications lead to unintentional changes in the production environment (see also C4). Fourth, ML components' high entanglement and eroded boundaries often lead to more complex dependencies in ML applications (see also C5). In sum, these challenges complicate ML application management and make it hard to monitor ML applications [58], [74], [75], [76].

There are few ML monitoring practices, and both measuring and deriving the appropriate conclusions remain challenges for organizations [77]. Currently, ML monitoring focuses on measuring different technical metrics, owing to the ever-present risk that new issues (e.g., concept drift or data exceptions) may occur in a production environment [13], [22], [57], [74]. In this regard, it is often unclear how changes in ML applications relate to organizational and business implications. There are possible determinants in a production environment that can affect an ML application's performance but that are not reflected in known metrics [78]. Yet capturing such determinants, drawing appropriate conclusions, and understanding arising tradeoffs are indispensable for an organization to permanently maintain an ML application's optimal functionality. Since the entanglement and impacts on a business layer are often not directly measurable [78], an ML application and the monitoring need careful design so as to help an ML application to work sustainably in an intended way [12], [59]. Particularly, one should consider that decision-makers outside the ML domain often have a limited understanding of the currently used technical metrics and have difficulties translating the utilized technical metrics into business metrics [77].

However, the research into monitoring ML applications is just beginning to understand what is important when monitoring ML applications [47]. Most studies on productive ML applications refer to the entire ML lifecycle (i.e., data processing, model training, deployment, and monitoring) [11], [79], [80] have treated monitoring only superficially

or not at all, and have provided no specific guidance on monitoring. As the counterpart to DevOps, incumbent firms such as Microsoft, Amazon, and Google have begun to advance a new approach called ML operations (MLOps). MLOps help organizations streamline their end-to-end ML workflows into production-ready applications [22], [76], [81], [82], but also provide little guidance on monitoring. For example, Amershi et al. [58] collected best practices for software engineering in ML from practice, but focused on application development. Breck et al. [4] analyzed ML applications' production readiness, presenting 28 test needs for ML applications, seven of which relate to monitoring, but they are only starting points. Zhang et al. [83] focused on testing ML applications and provided a detailed literature-based review of ML testing techniques. The test properties they identified may be appropriate starting points for metrics in monitoring. While Siebert et al. [84] derived quality criteria from an industry case and identified which metrics can be used to measure these criteria, they emphasized that the quality criteria and metrics refer to their case and that we still lack generally applicable criteria.

In the field of ML monitoring, there are a number of studies that provide an initial overview of the monitoring of productive ML applications, limiting themselves to technical components and corresponding tools [43], [51], [53], [55], [56], [60], [64], [85], [86]. Furthermore, one specific literature stream deals with the development of drift detection systems [17], [18], [61]. Generally, the – fairly technical – studies on ML build a valuable foundation to build on and investigate how organizations should monitor ML applications so as to maintain and increase their business value creation as well as to minimize their risks.

III. RESEARCH METHOD

To answer our research question, we followed a rigorous qualitative mixed-method research approach, integrating both scholarly insights and practical expertise in ML monitoring

in production environments [19], [25]. Therefore, we set out to conduct a multivocal literature review analyzing relevant peer-reviewed academic articles as well as grey literature (i.e., practitioner articles) on the monitoring of ML applications, to capture the latest status of ML Monitoring. Complementing our insights from the literature, we conducted an interview study allowing for first-hand insights from ML experts. Lastly, we analyzed existing ML monitoring tools to compare the collected data from the interview study and literature studies with the real-world instantiations of ML monitoring (See Figure 2 for an overview of the data we used). By deeply immersing ourselves in ML monitoring practice across industries through interviews, literature studies, and ML monitoring tools, we gained access to novel knowledge on ML practices.

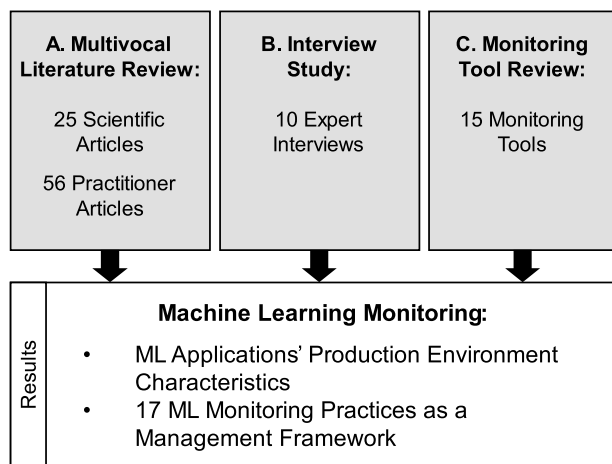


FIGURE 2. Collected data adapted from Kreuzberger et al. [20].

A. MULTIVOCAL LITERATURE REVIEW

To fully encompass the current discourse on ML monitoring, we conducted a multivocal literature review [29], [30] that included both academic literature and grey literature (e.g., practitioner articles). Ensuring rigor in our literature review, we followed the guidelines of Rudowsky et al. [30] as well as Russell and Norvig [29].

For the academic literature, we selected the databases IEEE Xplore and Association for Computing Machinery (ACM) as they comprehensively cover the current discourse on aspects such as state-of-the-art developments in ML applications and systems relevant to the implementation and evaluation of monitoring frameworks for ML. Additionally, we supplement the works from the two databases with publications from arXiv, an open-access repository for scientific preprints. This allows us to ensure that the most up-to-date research aspects, which have not yet been published in journals, are also incorporated into our findings. Derived from our research question, we used the search string “machine learning AND monitoring”. The search string ended up with an initial sample of 5887 articles in IEEE Xplore, 399 articles in ACM,

and 1473 articles in arXiv. To screen the initial sample, we excluded articles that were neither conference papers nor journal articles and did not have a dedicated focus on ML monitoring. In this manner, 62 articles remained after the title screening, 32 articles remained after the abstract screening, and 25 articles were included in our final sample after the full-text screening. For a list of the final scientific literature sample, we refer to Appendix A.

For the grey literature, we selected medium¹ as a database to have a database with various articles and ensure reliability in the sources. Medium¹ is an online publishing platform for social journalism. For instance, towardsdata-science² uses medium as a platform to share knowledge about data science through independent authors. We used the search string “machine learning AND monitoring”. To properly assess the identified articles, we applied the quality assessment checklist for secondary literature introduced by Garousi et al. [30] and incorporated it into our exclusion and inclusion criteria. We excluded articles if they did not focus on ML monitoring. We only included articles where the authors had a background in data science or ML engineering. We then also checked the articles’ structure to verify that their content is trustworthy and reliable. After applying the exclusion and inclusion criteria, we conducted a backward search on the identified articles. This resulted in 59 articles after the initial search. Eliminating nine articles based on the exclusion and inclusion criteria and adding six through the backward search led to 56 practitioner articles (hereinafter denoted as A01 to A56). For a list of the practitioner articles, we refer to Appendix B.

B. INTERVIEW STUDY

The interviews helped us better understand the challenges of monitoring productive ML applications and current best practices [87]. In line with the guidelines of Myers and Newman [87] as well as Schultze and Avital [88], we conducted ten semi-structured interviews with experienced ML monitoring practitioners. Data scientists and ML engineers emerged as key candidates for interviews since they are commonly responsible for the monitoring of ML applications [25]. We recruited our interview experts through the authors’ professional networks as well as through ML conferences. Since the last two interviews yielded no new insights beyond those already identified in the academic and grey literature as well as in the previous interviews, we concluded that, after a total of ten interviews across various industries (see Table 2), a saturation of insights has been achieved.

We structured the interviews along three key dimensions: the general challenges associated with ML monitoring, the technical aspects, and the organizational considerations. To encourage comprehensive and unbiased responses, we pri-

¹<https://medium.com/>

²<https://towardsdatascience.com/>

TABLE 2. Overview over the interviewed experts.

No.	Position	Experience (years)	Industry	Country	No. of employees	Min.
E01	Senior data analyst	<5	Software	Germany	<50	56
E02	CEO	>10	Startup ML monitoring	USA	<50	57
E03	Lead data scientist	>10	Healthcare	Germany	<50	55
E04	ML engineer	>10	Energy sector	India	<200	50
E05	Data scientist, product manager	>10	Consulting	Australia	>100,000	55
E06	Head of data science	>10	Retail	Germany	>4,000	51
E07	Data scientist	>10		Germany		
E08	Data scientist	>10	Software	Italy	<50	56
E09	Data scientist	<5	Healthcare	Germany	<50	50
E10	Data scientist	>10	Software	Germany	<10	35

marily asked open-ended questions, allowing the experts to freely express their insights without being constrained by predetermined categories, thus maximizing the richness of the data collected [25], [87], [88].

For data analysis, we rigorously used the coding techniques of Corbin and Strauss [89] and Saldaña [90]. One author extracted statements from the transcribed interviews, assigned descriptive codes to the statements, and labeled the statements as a challenge (i.e., interviewee mentions challenges in a monitoring practice) or practical approach (i.e., interviewee mentions a current monitoring practice). Thereafter, we conducted an axial coding round, and the author from phase 1 specified clusters in the interviewees' statements to better understand the interview data [90]. In several evaluation meetings among the authors, we formed categories and linked the statements to them [89], resulting in 385 codes and 31 categories.

C. MONITORING TOOL REVIEW

In order to obtain further data for a deeper understanding of ML monitoring, we collected data on existing ML monitoring tools in a third data collection step. We referred to the ML monitoring tools mentioned in the interviews and the practitioner articles. This resulted in a total number of 15 ML monitoring tools (hereinafter denoted as T01 to T15), which we present in Appendix C with a short description. To obtain the necessary information about the ML monitoring tools, we analyzed the documentation of the tools and publicly available information on the publishers' websites. During the analysis of the ML monitoring tools, we followed

the coding techniques of Corbin and Strauss [89] and Saldaña [90], considering the data analysis recommendations of Garousi et al. [30]. In an open coding round, we searched the documents for the main technical components of the ML monitoring tools and provided them with codes. In an axial coding round, we then clustered these components. In the following selective coding round, we compared the clusters with the ML monitoring practices: First, we enriched the existing ML monitoring practices with the lessons learned from the ML monitoring tools; second, we identified specific gaps in the ML monitoring tools related to the ML monitoring practices which we further discuss in Section V.

IV. PRACTICES FOR MONITORING MACHINE LEARNING APPLICATIONS IN THEIR PRODUCTION ENVIRONMENT

We arranged the monitoring practices along five steps of a typical quality management cycle to sequentially order them and ensure that they are complete. The cycle consists of five steps: define, measure, assess, act, and control [91], [92]. Every step comprises several monitoring practices. The define step contains monitoring practices that guide organizations in identifying the relevant problem areas and improvement areas as well as defining the appropriate metrics to measure these areas for all participating stakeholders. The measure step contains monitoring practices that help organizations collect the required data and measure the relevant metrics regarding the problem and improvement areas from the define step. The assess step contains monitoring practices that help organizations analyze the collected and measured metrics. These practices guide organizations in investigating causes, impacts, and other influencing factors in the problem and improvement areas, so as to derive appropriate conclusions. The act step contains monitoring practices that underpin organizations' tackling of the identified issues from the assess step by inducing adaptations. Finally, the control step contains monitoring practices that guide organizations in verifying adaptations made in the act step and in transferring necessary information back to the define step for further monitoring cycles. We provide an overview of the presented monitoring practices in Table 3. Columns C1 to C5 indicate the described influence of the five ML production environment characteristics on the monitoring practices.

A. DEFINE THE RELEVANT PROBLEM AND IMPROVEMENT AREAS AND THE ASSOCIATED METRICS (STEP 1)

Monitoring practice 1 in the define step is the **explicit definition of an ML application's weaknesses and compensatory workflows for those weaknesses**. The activities in this practice are well understood, especially in quality management [93], [94]. Owing to the ML algorithm's property of learning from a representation of data (C1), a certain percentage of errors (e.g., incorrect predictions) remains [43],

TABLE 3. Monitoring practices for machine learning applications.

Step	Monitoring practice	C1	C2	C3	C4	C5
Define	Define the ML application's weaknesses and compensatory workflows	x				
	Select appropriate metrics from the range between technical metrics and organizational ones					
	Model a metrics system for the selected technical and organizational metrics		x			x
Measure	Collect metadata from the ML application's context					
	Collect the ground truth label, if it is available	x				
	Collect metrics data for the ML application					
	Process the collected data and metrics for assessment		x			
Assess	Investigate the collected data and metrics so as to identify data quality issues			x	x	
	Investigate the collected data and metrics so as to identify drifts				x	
	Conduct cause-effect analysis for identified data quality issues and drifts					x
	Determine adaptations for identified data quality issues and drifts				x	x
Act	Communicate adaptations to stakeholders					x
Control	Verify the adaptations made in the ML application from the act step				x	x
	Transfer the required adaptations in the monitoring process to the define step		x		x	
Cross-sectional	Apply proactive mechanisms		x			
	Learn iteratively and continually	x				x
	Design monitoring tailored to use cases					

[45], [95] [E02; E06; A05]. While it is common for software applications to go live in a faultless state, for ML applications, it is necessary to consider such error types a priori [E09; A24]. It is crucial to clearly define these weaknesses and how they could burden the end-user [43], [45], [95] [E02; E08; A18]. An example from the quality management field for this practice is Failure Mode and Effects Analysis (FMEA) [96]. Thus, when problems arise owing to an ML application's weaknesses, a monitoring entity is able to take appropriate actions faster [43], [45], [95] [E02; E05; A05; A23; A47; A48; A52; T1; T8; T12]. Expert 02 confirmed this practice's importance: *"Have a good understanding of the classes with the types of weaknesses you need to look for and then, you know, matching for each of them, some sort of workflow. What would they be? How would they be addressed? What would they be? How would they feed into your plans?"*

Monitoring practice 2 is **select appropriate metrics from the range between technical and organizational metrics**. Activity 1 of the selection process is to cover the relevant information for all relevant stakeholders [18], [22], [39], [43], [45], [49], [50], [51], [52], [53], [60], [97] [E02; E04; E05; A09; A42; T1; T2; T4-12; T14; T15]. Computational metrics (e.g., latencies or CPU utilization) are more relevant for the operations team [A01; A24; A33]. Standard ML metrics (e.g., accuracy or F1 score) may be relevant for the data scientists from the application development team [E01; E03; E07; A01; A02; A28; A29; A40; A44]. Business-driven metrics are mainly of interest for the departments in which the applications are deployed [E02; E07; A07; A28; A33; A42]. Strategic metrics in the direction of business value are primarily in the focus of the management [E02; A28; A33]. The more technical metrics, i.e., computational metrics and standard ML metrics, often depend on the ML system used

(e.g., TinyML or on-device ML), which a monitoring entity should also consider when selecting metrics. The second activity is to derive metrics from an organization's business objectives [14], [22], [39], [43], [44], [49], [52]. Customer-centricity should be at the forefront of the application, both in development and monitoring [43], [E05; E06; A02; A05; A33]. Despite the many measurement options, the metrics selection should help an organization to align with its customers' business value-driven expectations [E02; E05; A09; A54]. Expert 05 addressed this aspect as follows: *"I think the most important metrics, are the ones that most closely reflect what's important to the client."*

Monitoring practice 3 is **model a metrics system**, which is generally also a known approach in software monitoring. While individual measurements capture only a part of reality (C3), collective metrics systems bring together various individual metrics, particularly technical and business metrics [14]. A critical aspect of any metric system is to define preferences [14], [44]. Accordingly, eliciting these preferences over metrics and objectives in a collective metrics system specifies the tradeoff space. Thus, metrics systems not only form an individual metrics set, but also help practitioners to understand their interrelationships [E02; E04; E06; A13; A18; A27; A28; A32; T2; T4-6; T10; T11; T14; T15]. Such a coherent understanding is crucial because, on the one hand, measuring only technical metrics does not help determine to what extent an ML application creates value concerning an organization's goals [E05; E08; A09; A28; A52]. On the other hand, waiting until issues unfold their impacts on business metrics is too late, since negative impacts may already have unfolded their full effects [E02; E05; A07; A09; A22]. For instance, in a predictive maintenance case, it may be insufficient to only know false positives and false negatives; it is helpful to also know the costs caused by false

positives and false negatives [98], [E08; E09]. Knowing such interrelationships supports monitoring entities in the assess step to understand a deviation's impacts on the false positive/negative rate [E07; A21; A44]. Expert 05 emphasized this: *"But we can at least kind of ask them. Okay, like, which metric is important to you. Like, how do you actually measure this, and what's going on here and how much time you usually spend and how much money that adds up to, and so on?"*

B. COLLECT AND PROCESS THE RELEVANT DATA AND METRICS (STEP 2)

Monitoring practice 4 in step 2 is **collect metadata from the ML application's context**. This is a well-known practice in SE and there are several tools that support teams in automatically collecting metadata for software monitoring [70], [A01; A20; A30]. Collecting metadata around an exception helps to identify the reason for such an exception [71], [E02; E04; A10; A16]. Typical examples of metadata are bug tracks, parameter configuration, time-stamp fields, hostname fields, or versions [70], [A10]. First, log collection is as crucial for ML applications as it is for software applications [39], [45], [50], [53], [55], [57], [99], [100], [E09; A04; A06; A20; A27; A33; A51; T1-4; T9]. Second, we emphasize that particularly ML versioning is more complex than software versioning owing to the high entanglement between ML algorithms and data (C5). Besides code versioning, it is indispensable to version the used data, model, and hyperparameter setting [18], [22], [39], [44], [45], [49], [50], [53], [57], [82], [95], [E09; A08; A11; A27; A33; A34; A38; A52; T1-4; T9]. Third, metadata in an ML context are an ML algorithm's inputs and outputs that may hold important information for assessing an ML application's working status [39], [50], [53], [55], [56], [57], [101], [E01; E02; A06; A07; A37; T1-4; T9]. According to Expert 02, a monitoring entity should collect *"everything that happened along that session, all the metadata around that user session, who they are, where they come from, and so on. Inputs into the model, the outputs of the model, and the outcome."*

Monitoring practice 5 is **collect the ground truth label, if it is available**. Because ML learns from data (C1), the ground truth label is a highly ML-specific characteristic. Activity 1 before tracking the ground truth label is to evaluate the ground truth label's availability by considering three aspects [17], [39], [43], [55], [56], [57], [E02; E05; E07; E08; A17; A22; T2; T4-6; T10; T11; T14; T15]: First, there may be time delays between the prediction and the occurrence of the ground truth label [57] [E02; E08; A28; A37]. For instance, predicting a credit fraud means that the de facto fraud occurs weeks later, and the ground truth is also only available at this time [A04; A15; A16; A37]. If a use case requires real-time feedback from the monitoring system, the ground truth is not useful, but a monitoring entity can use it for a post-evaluation in a later stage [E02; E09; A15; A38].

Second, an intervention based on an ML application's prediction may lead to a changed reality, so that an appropriate ground truth to verify the prediction will never be available [E05; E09; A49]. For instance, an ML application predicts customer churn, and the organization initiates activities to prevent such churn [A04; A28]. Later, when the corresponding customer does not churn, no ground truth is available that tells the organization if it was because of its actions or a wrong prediction. Third, data samples may exist that need to be labeled which entails certain costs [E02; A02; A05; A08; A24]. For instance, in many computer vision tasks, humans need to label the images [A04]. Activity 2, if the ground truth label turns out to be useful for the use case, is to implement the labeling mechanisms [17], [39], [44], [51], [52], [55], [56], [57], [60], [64], [95], [97], [101], [E02; E05]. For instance, human-in-the-loop labeling mechanisms are a useful source for generating ground truth during monitoring [E05; E08; A07; A14; A16]. But such incorporation of end-users should not burden them too much during the labeling activity: *"The point is that if the client has to check too often [...], then it's no longer useful, because they want something to forget about. And so you have to be sure that you ask the client to check only what [...] is really not clear for you."* [E08].

Monitoring practice 6 includes the **collection of the metrics data for the ML application that were defined in practice 2 in the define step**. First, owing to the different metric types in ML applications, organizations must ensure that they collect all the required metrics data to calculate or receive the metrics with the use of an adequate infrastructure [39], [43], [50], [56], [57] [E02; E09; A01; A20; A22; T2; T4-6; T10; T11; T14; T15]. Business metrics often sit in business intelligence systems. So, to get these metrics, ML monitoring entities must dock the business intelligence system to the ML monitoring system [E02; E07; E09; A22]. Computational metrics are often available in software operations, and ML monitoring entities may use the same tools for these metrics [E02; E09]. Standard ML metrics often need the ground truth (see category 2 in the measure step) to be calculated; thus, the calculation should have a connection to the ground truth labeling mechanism [E03; E06; E09; A49]. Second, for all metric types, it is necessary to measure them in a useful interval [43], [44], [49], [53], [57], [60], [82], [97], [101], [A28; A32; A42; A53]. The interval should not be too long so as to avoid having to process too many metrics data (see practice 4 in the measure step) and to avoid ending up with an over-engineered monitoring system [E08; E09]. Yet the interval should also not be too short so that no important information is lost [A28; A42].

Monitoring practice 7 is **process the collected data and metrics from the previous categories to prepare them for the assess step**. Since the ground truth label (see practice 2 in the measure step) is available only in specific circumstances, ML monitoring needs to focus on new ways to measure an ML application's working status (C2). The first option is to

join related metadata (from practice 1 in the measure step) and the ML application components in question to gain new insights into an ML application [50], [53], [55], [56], [57], [101], [E03; E04; A01; T1; T2; T4-7; T9-12; T14; T15]. The second option is to use statistical calculations for an ML algorithm's inputs and outputs [18], [43], [50], [56], [60], [64], [E02; E03; E05; E08; T1; T2; T4-7; T9-12; T14; T15]. A monitoring entity can then extract information from that newly generated statistical metadata, which they can use for approximations and relative assessments of an ML application's working status in the assess step [43], [E05; E08]. On the input side, such measurements may be the percentage of missing values, statistical moments, quantiles, or statistical distributions [E02; E08; A12; A15; A24; A35; A42; T1; T2; T4-7; T9-12; T14; T15]. On the output side, experts often refer to behavioral metrics to measure an ML application's behavior in the production environment through its predictions [E02; E05; E08; A17]. Examples of the output side include measurements such as class and cluster distributions, confidence scores, or distances between decision thresholds [E08; A08; A16]. Expert 07 underlined the importance of statistical measurements in ML applications: "You can then compare debt fraud or the activity that you classified as that fraud; you can compare it to the distribution of the activities that in the past were actually confirmed as a fraud."

C. ASSESS CAUSES, IMPACTS, AND OTHER INFLUENCING FACTORS (STEP 3)

Monitoring practice 8 involves **investigating the collected data and metrics to identify data quality issues**. While assessing metrics is a basic task in software monitoring, additional data assessments are required for ML applications, since data contain several assumptions (C3) and changes over time (C4). Activity 1 includes the detection of data quality issues in data processing, which an application may be unable to process [14], [39], [43], [44], [45], [97], [102], [E05; E08; A12; A22; A25; A45; T3; T5; T7-9; T11-13; T15]. An example of data quality issues is investigating missing and not a number (NaN) values in the production data [A02; A25; A47; A51]. If the missing or NaN values increase for certain features, data entry may not work appropriately [A09; A12; A47]. A second example is a deviation in a feature's value range, which could be because the feature's unit is different (e.g., switching from Celsius to Fahrenheit) [A02; A05; A25]. A third example is anomaly detection in production data, i.e., a monitoring system should extract outliers [A01; A19; A21; A36; A45]. Activity 2 includes the necessity to check collected metrics against the state of the metrics in the training phase or various thresholds [17], [18], [49], [56], [60], [82], [97], [101], [E06; E09; A05; A07; A12; A16; A18; T3; T5; T7-9; T11-13; T15]. For instance, a monitoring entity should regularly compare the fairness metrics' state of the training phase to the state of the production phase

so as to swiftly detect upcoming fairness issues [20], [A14; A16]. Activity 3 includes checking all the data and metrics not only for quality issues but also for key data slices [39], [61], [97] [E01; E09; A05; A16; A18; A22; A23; A33]. A metric may have an overall value that is acceptable but may have an unacceptable value on a specific subgroup [39], [61], [97], [A04; A07; A10; A19; A21; A24; A26; A28]. Expert 01 outlines this aspect with the following example: "Checking how [the ML application] performs in maybe several subgroups? Does it work better on men than on women or something?"

Monitoring practice 9 is **investigate the collected data and metrics to identify (adversarial) drifts**. Drift detection is a highly ML-specific characteristic since changes in ML application often mean that the training data deviate from the production data (C4) [12], [43], [59]. The monitoring system should help to identify changes in the input, output, and the relationship between input and output [17], [18], [22], [39], [43], [44], [45], [49], [50], [51], [53], [55], [56], [57], [60], [61], [64], [97], [E04; E09; A01; A15; A23; A39; T5; T7-9; T11-13; T15]. Expert 04 confirmed this: "Models drift over time, due to seasonal changes, training on limited datasets, or data pipeline issues. Hence, it would be in the interest of operationalization to put a drift monitoring system in place." Otherwise, the ML application's prediction begins to diverge from the desired state and becomes unreliable [E01; E02]. First, during drift detection, it is crucial to differentiate between the drift types concept drift (i.e., $p(c|X)$ changes and $p(X)$ not), data drift (i.e., $p(X)$ changes and $p(c|X)$ not), and virtual drift (i.e., $p(c)$ changes and $p(X|c)$ not) [17], [18], [61], [A01; A02; A23; A36; A47; A41; A55]. The drift types are reflected differently in the underlying data and may have various implications [E02; E04; A47]. Drifts can occur in several ways, for instance, suddenly, as through the Coronavirus pandemic, or fairly nuanced, through slow changes in the production environment [E02; A10; A44; A47]. Moreover, adversarial drifts might result from maliciously injecting data [103], [104]. Second, to discover drifts, the monitoring entity should verify whether an ML application makes its prediction in the production environment on the same underlying distribution as in the training phase [E02; E09; A01; A04; A21; A22; A23; A24; A25; A28; A29; A30; A31; A33; A40; A43; A50; A54; A56]. Using the statistical measurements from practice 7 helps ML monitoring entities to make the necessary comparisons between the training and the operation status. For instance, a strong divergence in the prediction's distribution may indicate a change. Such a divergence needs further investigation of the causes in the following practice.

In monitoring practice 10, a monitoring entity should **conduct a cause-effect analysis of the identified data quality issues and drifts in the previous practices**. Cause-effect analysis is not a new endeavor for software monitoring, since any software issues require addressing the root cause, not the symptoms, and tracing the issues' impacts on the

application's objectives [105], [106]. However, ML's black box character creates hidden entanglements (C5) that often impede assigning a failure to an ML application component [75]. We divide this practice into three tasks: First, also in the ML context, it is indispensable to track the dependencies between different ML application components over time, to learn the relationships in an ML application, and to assign the causes to the source of the failure in question [22], [44], [50], [53], [95], [E02; E03; E04; A01; A03; A05; A07; A09; A12; A13; A14; A15; A16; A17; A26; A34; A36; A51; T1-5; T8; T9; T12; T13; T15]. For instance, in some cases, explainability methods (e.g., SHAPE or LIME) can generate more human-interpretable and deeper insights into what dependencies in an ML algorithm may look like [22], [95], [A01, A14, A16]. Second, for all causes, one should differentiate between model faults and external exceptions so as to derive appropriate actions in the next monitoring practice [51], [E02; E04; E05; A12; A42; T1-5; T8; T9; T12; T13; T15]. This is especially important if a model has been poisoned, targeted for adversarial attacks, or reacted in a way that produces content that is not appropriate for certain groups or cultures, for example. Third, it is crucial to determine the causes' impacts on the key business metrics so as to logically prioritize the order of the adaptations [14], [22], [39], [44], [49], [50], [51], [53], [57], [61], [E01; E02; E04; E06; E07; A05; A19; A21; A26; A44; A52; A53; T1-5; T8; T9; T12; T13; T15]. For this case, the metric system from the step define step may be useful. The importance of determining discovered causes' impacts on the business level emphasizes Expert 02's metaphor: *"The holy grail is to find new connections, to find... new impacts on [business] KPIs."* For all three tasks, the collected metadata from the measure step provides support. For the detected quality issues, a monitoring entity should use the metadata around the features in question or data points to diagnose, for instance, the causes of an outlier or a drift [18], [63], [E01; E02; E04; A01].

Monitoring practice 11 is **determine adaptations for the causes of the identified data quality issue and drift**. While this task is also common in software monitoring, we note that assumptions (C3) and the high entanglement in ML applications (C5) complicate the mutual influence of adaptations and comparisons with prior adaptations. In activity 1, the monitoring entity should refer to prior versions of the ML application [18], [22], [39], [44], [45], [49], [50], [53], [57], [82], [95], [101], [E01; E04; E05; T5; T8; T9; T12; T13; T15]. It is only with version information (e.g., used data, features, and hyperparameter) that a monitoring entity can reasonably compare adaptations or initiate rapid rollback when an urgent issue occurs [53], [57], [E04; E09; A13; A27]. In activity 2, when determining adaptations for mitigating the identified quality issues, a monitoring entity must make tradeoffs, since it is not possible to implement every adaptation without influencing another adaptation [44], [E01; E02; E05]. This also means considering how a tradeoff may influence different stakeholders [44], [E01; E02; A22].

For instance, Haas [107] investigated how a tradeoff between the accuracy metric and fairness metrics can be handled, since focusing on accuracy means a lower fairness, and vice versa. In the case of Expert 01, the prioritization was as follows: *"And if we had a lower performance but [the output] was explainable, then we took this model, so this was in any case the condition and also, yes, like a bit of the own ethical claim."* In activity 3, there is the necessity to determine an ML algorithm's ideal retraining time [18], [22], [39], [44], [50], [57], [82], [95], [97], [E01; E02; E03; E04; E06; A11; A22; A47; T5; T8; T9; T12; T13; T15]. It is not possible, as in SE, to fix and redeploy the corresponding software code. Instead, an ML algorithm needs complete retraining which is associated with correspondingly required computer resources, incurring costs [A15; A54]. Depending on the use case, retraining may only be possible in certain periods when the required new data are available for retraining or may be even critical because the ML algorithm has been poisoned or has produced discriminatory content [E01; E02; E07; A11; A15; A30; A38]. In activity 4, based on the identified causes and effects from the previous monitoring practice, a monitoring entity should decide which adaptations (e.g., retraining or model substitution) in the ML applications are essential [18], [22], [39], [44], [50], [57], [82], [95], [97], [E01; E02; E05; A05; A18; A22; A35; T5; T8; T9; T12; T13; T15]. For this, they should orientate on the organization's objectives and the identified business impacts of a quality issue or drift [49], [61], [E01; E02; E06; E09; A10; A41]. Since not every issue is worth being optimized, a monitoring entity can prioritize necessary adaptations based on the orientation on the business impacts and can generate a roadmap for the adaptations [E02; E05; A10; A23; A35; A41; A54].

D. ACT TO REALIZE IMPROVEMENTS AND TACKLE ISSUES (STEP 4)

Monitoring practice 12 deals with **communicating the adaptations to the stakeholders**. Bringing new insights that a monitoring entity gained from the previous step assess back to the development department is a core idea in DevOps [71]. In activity 1, a monitoring team should communicate the necessary adaptations to the development entity of the ML application [53], [E04; E06; E07; A09; A27; A40; A50; T5; T8; T12]. In activity 2, the monitoring entity should also communicate the adaptations to the application's stakeholders [43], [53]. Communication about failures, adaptations, and their impacts must be done in a consistent language with the affected stakeholder [39], [43], [53], [E01; E02; E04; E06; A20; A23; A34]. Particularly, dashboards and visualizations are helpful ways to provide all stakeholders with the appropriate information [8], [E05; E08; E09; A34; A44; A46; T5; T8; T12]. However, owing to ML's high data dependency (C1) and the black box problem (C5), many stakeholders need additional understanding of the interpretability of measurements or how to deal with

errors by an ML application [E01; E05; E08; E09; A05; A22; T5; T8; T12]. Since data literacy and ML knowledge remain low in many companies across many stakeholders, the monitoring entity should establish suitable communication channels [E03; E08; A05]. Expert 04 emphasized this problem and the need to address it: *“Education for the whole organization is necessary, for better understanding. Develop a language that everyone understands.”* When establishing communication channels with the stakeholders, it is crucial to enhance process transparency [44], [48], [71]. Because of the paradigm change in developing ML, ML applications have fewer rules-based entanglements (C5). Thus, it is important to create transparency around how ML applications are working, both on the training level (e.g., learning own rules based on data) and the operation level (e.g., new pipeline types) [44], [E01; E02; E04; E06; E08; A09; A47].

E. CONTROL THE PRECEDING STEPS IN THE LONG TERM (STEP 5)

Monitoring practice 13 is **verify the adaptations made in the ML application from the act step**. This is generally a well-known practice in software monitoring. Owing to many changes in an ML application’s production environment (C4) and the high entanglement (C5), there is no certainty in an ML context that adaptations made will result in the intended improvement [75]. As in monitoring practice 4 of the assess step, a failing adaptation can become an urgent issue, and rapid rollback may be required. To verify the adaptation, it is useful to give the necessary information to the define step so that, in the next monitoring cycle, the ML monitoring entity can observe the adaptation’s impacts [39], [44], [53], [E01; E04; E09; A18; A52]. Expert 04 emphasized the importance of this practice: *“Although model performance is a subjective matter, newer models should be at least a couple of points greater than the ones deployed.”*

Monitoring practice 14 is **transfer the required adaptations in the monitoring process to the define step**. This is also not new for software monitoring since it is always possible to discover additional measurement (C2) possibilities in an application. In an ML context, this may be more common than in normal software applications, since there can be many changes in the production environment (C4) that cannot be foreseen by monitoring [59]. So, bringing all the learned insights during a monitoring cycle (i.e., running through steps 1 to 5) back to step 1 (i.e., define) helps to improve the monitoring approach [E04; E05; E09; A52].

F. CROSS-SECTIONAL MONITORING PRACTICES

During the coding of the interviews and practitioner articles as well as the clustering of the resulting codes, we stated that there are some cross-sectional practices. We identified another three guiding practices that should be applied to all

monitoring steps (1 to 5): **apply proactive mechanisms, learn iteratively and continually, and design monitoring tailored to use cases**.

Monitoring practice 15 is **apply proactive mechanisms**. This generally applies to all software application monitoring activities. A monitoring system should seek to capture possible issues as early as possible and not just react to unfolding issues [43], [45], [95], [E01; E02; E03; E09; A13]. The fact that performance in productive ML applications is not always directly measurable (C2) emphasizes the high need for proactive mechanisms in ML monitoring [44], [53], [78]. Thus, activities such as the selection of metrics, the assessment of insights based on the selected metrics, and the derived actions should always be proactive [44], [53]. However, there will always be aspects that a monitoring entity has not considered in advance. For both proactive and reactive mechanisms, the monitoring approach should allow for appropriate changes to the ML application in order to bring the system back to the defined optimal running state. As Expert 02 described monitoring’s role, *“Monitoring is meant to, in a way, shorten the response time you have to business issues.”*

Monitoring practice 16 is **iteratively and continually learn from the production environment**. The research has often mentioned continual delivery, continual deployment, or continual monitoring, among others, as key practices in DevOps. Since ML application’s data only represent the real world (C1) with many unforeseeable changes (C4), ML applications may need a significantly higher number of learning cycles. This requires a mindset shift from trying to create near-perfect working applications a priori since it is harder for an ML application to cover all possible cases in the production environment [12]. Instead, ML monitoring entities should establish a virtual cycle that enables ML applications to continually learn from the production environment, improve, and expand their scope [21], [22], [39], [44], [53], [57], [61], [82], [97], [E02; E04; E09; A13; A18; A33; A42]. For instance, an ML-based language translator should become better concerning accuracy and scope to match more different dialects (if in line with the objectives of the application and the organization) [E02; E04; A22; A29; A40]. This iterative, continual learning is also reflected in this statement by Expert 02: *“But the reason we [the monitoring entity] exist is to enable companies to learn from production. And they continue to improve.”*

Monitoring practice 17 is **design monitoring tailored to the use cases**. Building monitoring systems is a well-known practice since it is often at the core of current software monitoring research. Owing to the many possible measurement types, there is the need to tailor ML applications to the organization’s and use case-specific circumstances as well as the monitoring entity. First, ML monitoring needs a careful design, and off-the-shelf ML monitoring systems may not be appropriate enough to cover all the required aspects of

a specific use case or organization [82], [101], [E04, E06; E07; A07; A10; A17]. Since the developing entity best knows the requirements for the ML applications, it may be useful to develop an in-house monitoring system together with the ML development entity [E04; E05; E08; E09; A17; A52]. Second, owing to the many ML use case types and the many measurement possibilities, a monitoring entity should be modular and flexible to many necessary adaptations and iteration cycles [E04; E05; A07; A17]. Despite the many possibilities and things to measure, the monitoring entity should generally not be more complex than the application itself, as Expert 08 confirmed: *“And I think this is the overall rule in any monitoring system, because if this monitoring system is constantly sending alerts, there is something that is not working – either the monitoring system or the model.”*

V. DISCUSSION

Based on a wide range of practical insights, we have derived 17 monitoring practices that we assigned to a typical quality management cycle (define, measure, assess, act, and control). This quality management cycle is in line with our definition of monitoring and allowed us to bring the monitoring practice in sequential order so as to better survey the entire monitoring practice. Our results provide an overarching perspective on the steps required for the successful end-to-end monitoring of ML applications. In that way, we did not remove monitoring practices that would also apply to non-ML applications (e.g., practices in the design step).

In alignment with the conceptual distinction between the representation layer and the real-world layer, as discussed in Section II-A, it is crucial to recognize the inherent complexities in monitoring machine learning (ML) applications. The representation layer, which encapsulates the way in which ML models abstractly depict the real-world phenomena, faces significant challenges when it comes to accurately reflecting the complexities of the real-world layer. From a static perspective, the characteristics C1 (data representation), C3 (assumptions), and C5 (entanglement) complicate the appropriate and comprehensive representation of the real-world layer as they lead to the inherent issue that not every aspect from the real-world can be adequately depicted in the representation layer. From a dynamic perspective, C4 (changes) introduces additional complexity by acknowledging the ever-changing nature of the real world. What may be an appropriate representation at time t_0 may no longer be valid at time t_1 due to shifts in underlying real-world conditions. This dynamic perspective highlights the limitations of static representations and emphasizes the need for continuous adaptation of ML applications to remain aligned with the real-world layer. Furthermore, it is important to note that the representation layer is only accessible through the observation of metrics (C4) at the core measurement areas

in the ML application’s main parts (see Section II-A), which introduces the possibility of bias. An observer’s interpretation of an ML application’s performance and behavior can influence how the representation layer is perceived, potentially skewing the monitoring process. Bias in this observer layer complicates the task of ensuring that the representation remains accurate and unbiased over time. The ML monitoring practices, therefore, play a critical role in mitigating the complications posed by C1 through C5. Effective monitoring not only helps reduce the static and dynamic complexities associated with representing the real world but also allows for more accurate observation of the ML application’s behavior over time. By incorporating mechanisms to continually assess and adjust the alignment between the representation and the real-world layers, monitoring practices can help organizations ensure that their ML applications maintain appropriate performance and reliability in dynamic production environments.

Moreover, the number of practices (8) that focus on metrics indicates the importance of quantitative measurements in monitoring. Expert 05 emphasized the status of metrics in monitoring: *“For us, metrics-based monitoring has been a big part of kind of feeling us, feeling comfortable with things, doing what they’re supposed to do in the client, right? And the same thing for them.”* The research has mostly mentioned measurement and assessment activities as the main activities in monitoring [70], [71], [72], [73], [108], [109], [110]. This is also evident in our derived practices, since more than half the practices are attributable to the two steps measure and assess. Although the literature has often focused on measuring and software systems to measure, we state that organizations must not neglect the other nine practices, because they often involve crucial interface functions that enable the embedding of all the required stakeholders. Such stakeholder involvement helps, for instance, in metrics selection or appropriate ML application adaptation. Further, we identified three monitoring practices that are more generic. An ML monitoring entity should consider these practices as a cross-section that applies to the other 14 practices. They serve as an overarching recommendation on how to conduct monitoring in the individual steps. The practices apply proactive mechanisms and learn iteratively and continually are recommendations on how an ML monitoring entity should conduct the activities in steps 1 to 5. The practice design monitoring tailored to use cases is a recommendation on how a monitoring entity should design steps 1 to 5.

Nonetheless, the de facto implementation of the ML monitoring practices is nontrivial and does not correspond to a standard procedure. Instead, the practices’ implementation should consider the ML application’s embedding (e.g., on-device vs. in the cloud or TinyML vs. full-fledged data center). In the case of self-adaptive systems, certain practices can even be automated or deeply integrated into the ML application [111]. To ensure that the implemented moni-

toring practices help minimize risks and increase business value, these are crucial: the ML monitoring's integration into a whole SE process, the ML monitoring's integration in the whole ML lifecycle, and the definition of roles in an ML context. First, the derived monitoring practices help to successfully monitor an organization's ML applications. However, for successful ML use in production environments, there are more aspects to consider. For this, it is key to integrate the monitoring practices into the whole SE process and to connect them to other software monitoring and DevOps principles. Thus, we recommend further investigating merging in an SE and data science-based way – for instance, Amershi et al. [58], Arpteg et al. [12], or Lwakatare et al. [48]. Second, ML monitoring's embedding in a typical lifecycle is also necessary from a pure ML perspective. Particularly evaluation and testing approaches in ML development deal with metrics and should align with ML monitoring. Third, since there is much research into required skills, roles, and responsibilities in the DevOps literature [73], [112], these questions also arise in the ML field. Based on our derived monitoring practices and their discussion, we propose the following research opportunities in Table 4:

TABLE 4. Future research opportunities.

Results and Discussion	Derived Future Research
<i>Step Measure:</i> Collect the ground truth label if it is possible and useful	How should one design appropriate human-in-the-loop labeling mechanisms?
<i>Step Define:</i> Select appropriate metrics from the range between technical and organizational metrics	What are the required basic metrics to monitor?
<i>Step Define:</i> Model a metrics system for the selected technical and organizational metrics	How does one select metrics for ML monitoring in relation to the underlying use case?
<i>Step Assess:</i> Investigate collected data and metrics to identify drifts	Which drift types exist in ML and how should one measure them?
<i>Step Assess:</i> Conduct cause-effect analysis for identified quality issues and drifts	Which components make up a decision support system for ML tradeoffs?
<i>Step Assess:</i> Determine adaptations for identified quality issues and drifts	
<i>Step Act:</i> Communicate adaptations to stakeholders using the application	How does one bring more knowledge on and acceptance of ML to employees in an organization?
<i>Cross-sectional practice:</i> Design monitoring tailored to use cases	How should one design a monitoring system and an architecture for ML monitoring?
<i>Overarching results</i>	How does one integrate ML monitoring into an organization's SE processes?
<i>Overarching results</i>	What are the relevant roles, responsibilities, and required skills for ML monitoring?

VI. CONCLUSION

We have constructed a framework of productive ML applications to provide a better understanding of ML applications' working mode in their production environments, emphasizing ML monitoring's role in ensuring an ML application's intended working mode. To conduct rigorous ML monitoring, we derived 17 ML monitoring principles from an interview study and a multivocal literature review. To operationalize these practices, we propose further research into some of the practices.

Despite our comprehensive research approach, our study has limitations. First, ML monitoring is also an emerging task in practice; thus, only a limited number of interview partners were accessible to us. By achieving saturation in the interview responses and integrating multivocal literature on ML monitoring, we ensure the ML monitoring practices' completeness and depth. Second, using the multivocal literature could bias results toward a non-objective direction. We therefore rigorously adhered to the guidelines of Garousi et al. [27] as well as Ogawa and Malen [26] for analyzing the multivocal literature. Third, we focused on only one database of multivocal literature in data science and machine learning. By choosing an established platform with strict publishing guidelines, we have ensured correct and nonbiased content.

Our research has contributed to the recent academic discourse on AI management [19], [28], [29] and practice in several ways. First, the provided five characteristics that describe an ML application's production environment and the conceptualized framework of an ML application in a production environment expand the current discourse around ML with a concrete perspective from the operational use of ML. The derived five production environment characteristics and the framework can help organizations to better understand productive ML applications, the challenges that arise through an ML production environment, and monitoring's role in confronting these challenges. Second, we developed 17 ML monitoring practices and analyzed the influences of our five ML production environment characteristics on these ML monitoring practices. With this assessment of influences, we discussed how ML-specific characteristics affect monitoring practices.

The ML monitoring practices provide the required steps to ensure an ML application's functionality. For research, the practices may serve as a basis to investigate the implementation of single practices more deeply (e.g., implementing appropriate drift detection mechanisms). Further, our approach may help researchers to investigate ML applications from a production perspective and in coherent consideration with the existing SE research. The derived ML monitoring practices can guide organizations on what the critical steps are in conducting successful ML monitoring. Only the areas in an ML application that an organization monitors and measures can be continually sustained and improved.